

Ingenieria del Software

Tema 7. Despliegue

TABLA DE CONTENIDOS

01 **Despliegue**

02 **Silos**

03 **DevOps**

TABLA DE CONTENIDOS

01 **Despliegue**

02 **Silos**

03 **DevOps**

Despliegue

TABLA DE CONTENIDOS

01 Despliegue

02 Silos

03 DevOps

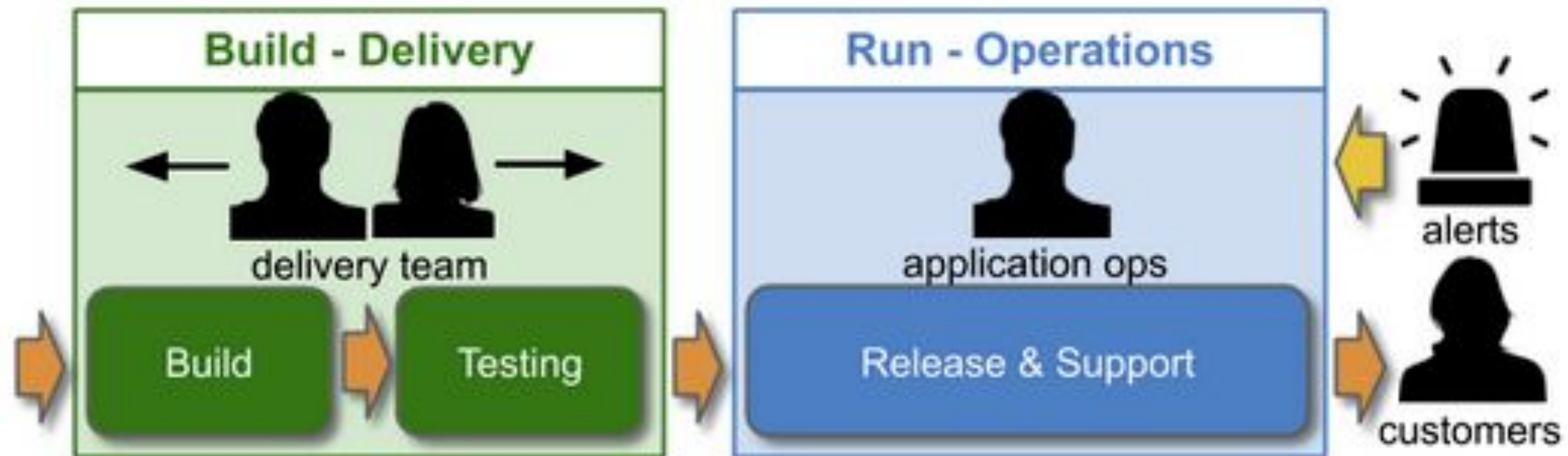
Silos

Despliegue

“El despliegue de software son todas las actividades que hacen que un sistema de software esté disponible para su uso.”
R. Pressman, Ingeniería de software

Silos

El **despliegue** de software son todas las actividades que hacen que un sistema de software esté disponible para su uso.



“you build it, ops runs it”

Modelo de Soporte

El estándar de [Operaciones de Servicio ITIL v3](#) establece un soporte multi-nivel atendido por diferentes tipos de especialistas.

Level	Team	Description	Support
1	Service Desk	Resolve simple customer issues in call centres, raise incidents, and often escalate to L2	N/A
1	Operations Bridge	Monitor dashboards, receive alerts, raise incidents, and always escalate to L2	24/7 Shifts
2	Application Operations	Monitor dashboards, respond to incidents, resolve incidents, and sometimes escalate to L3	On-call out of hours
3	Delivery teams	Resolve incidents for specific applications	Best endeavours out of hours

Modelo de Soporte

En función de su complejidad y proximidad al usuario, el soporte suele dividirse en:

- ❑ **Soporte de primer nivel.** Está en contacto directo con el usuario. Posee conocimientos generalistas y cuando no es capaz de resolver una incidencia la escala al soporte de segundo nivel.
- ❑ **Soporte de segundo nivel.** No interactúa con el usuario sino que se relaciona con él a través del soporte de primer nivel. Posee conocimiento específico y avanzado sobre las diferentes tecnologías que soportan los servicios. En caso de no ser capaz de resolver la incidencia ésta se escala al soporte de tercer nivel.
- ❑ **Soporte de tercer nivel.** Soporte especializado prestado por el propio equipo de desarrollo del producto software.

TABLA DE CONTENIDOS

01 Despliegue

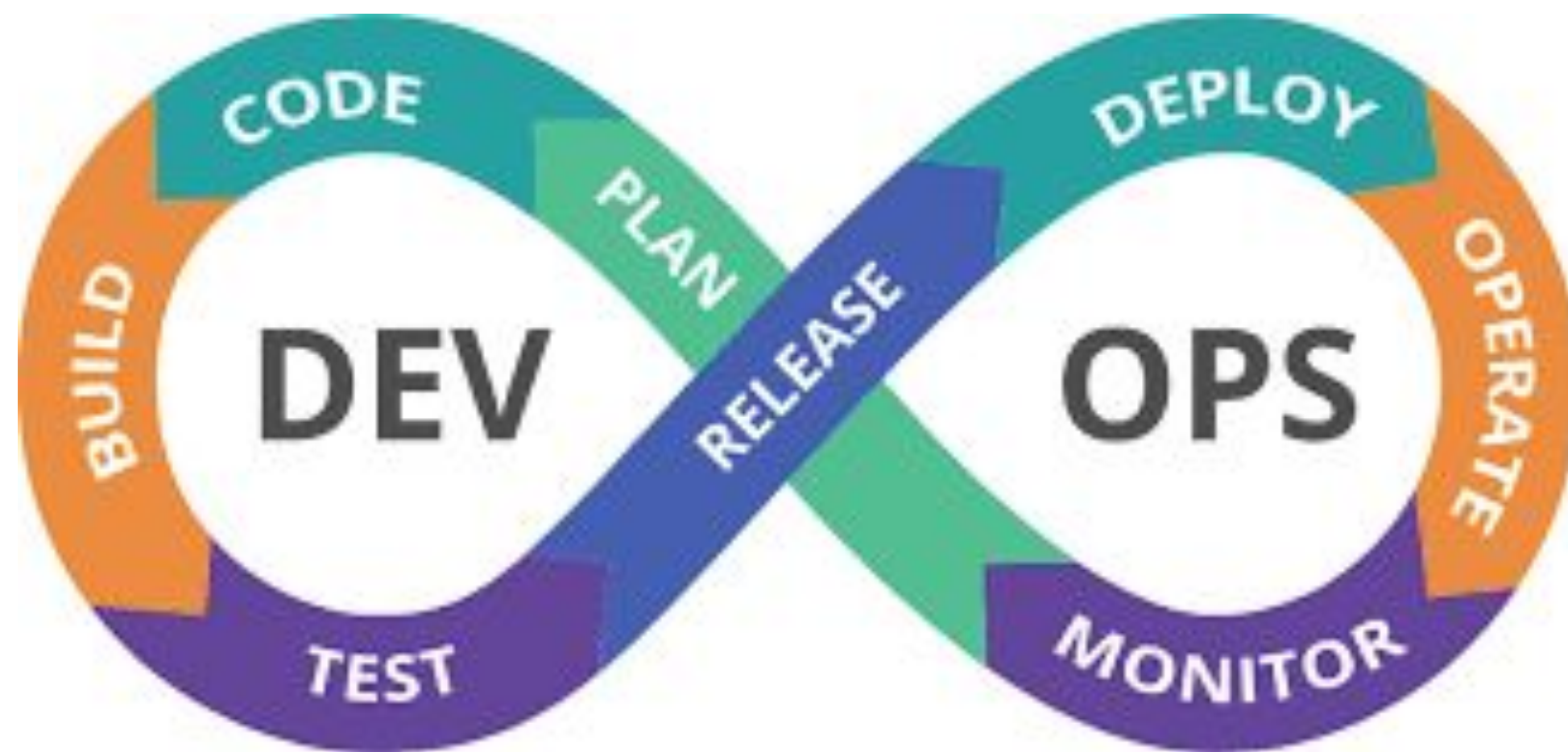
02 Silos

03 DevOps

Devops

¿Qué es DevOps?

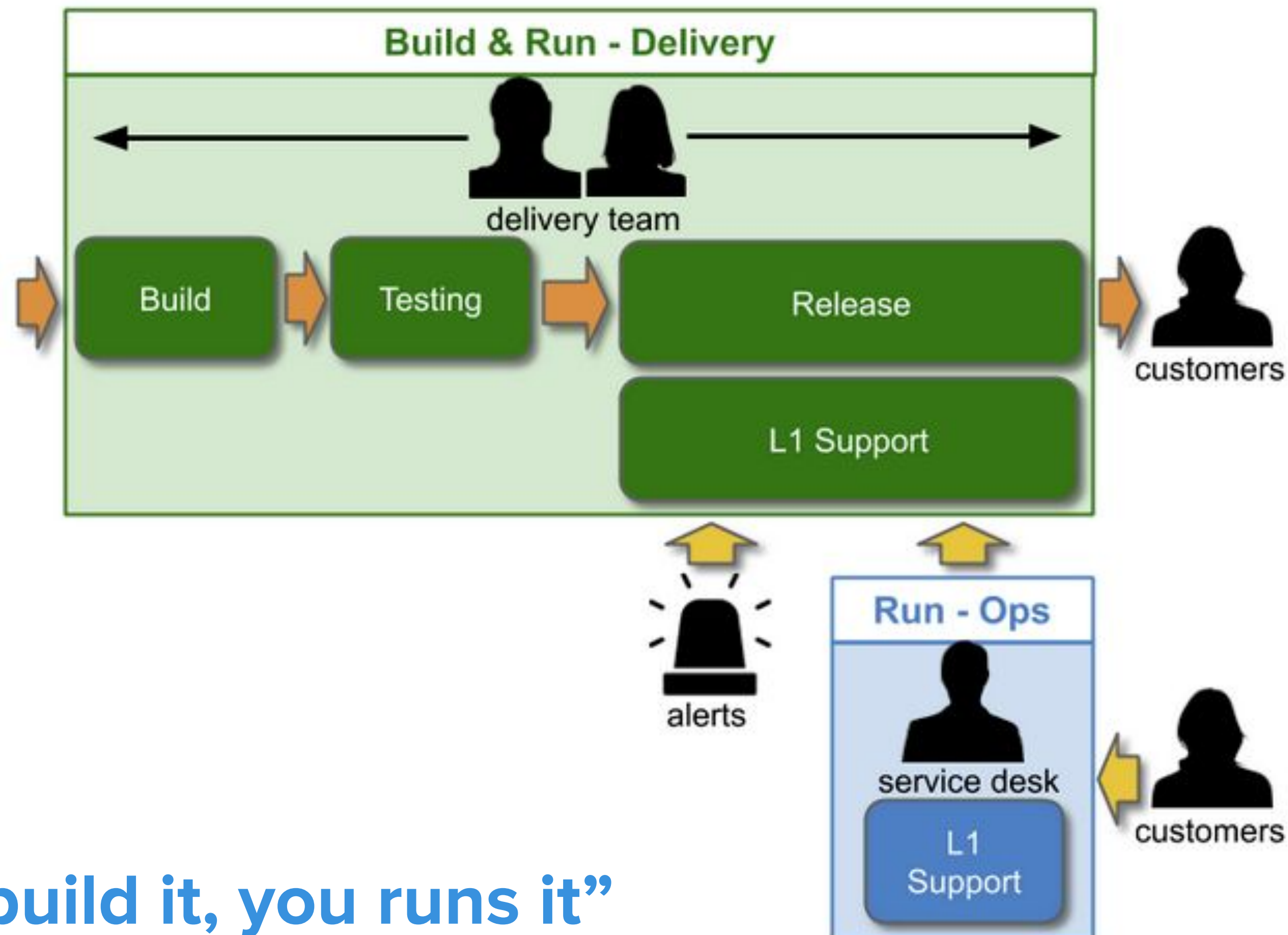
Los cada vez más exigentes plazos de entrega han provocado un considerable aumento del número de despliegues ([10 despliegues cada día en Flickr en 2009](#)). La estrategia “[you build it, Ops runs it](#)” basada en silos (equipo de especialistas en despliegues) no es eficaz para la entrega intensamente iterativa



Devops pretende eliminar el silo de “operaciones” difuminando las fronteras entre los equipos de desarrollo y de operación (despliegue). Devops es un **conjunto de metodologías que permite a los equipos de desarrollo y de operaciones trabajar de forma más cercana entre ellos**, aportando mayor agilidad al proyecto y notables incrementos de productividad. Pretende dar agilidad y responder a las necesidades del flujo de desarrollo, optimizando los procesos de desarrollo, tests, despliegue, resolución de bugs, paso a producción y monitorización.

¿Qué es DevOps?

Es el propio equipo de desarrollo quién asume la responsabilidad de desplegar su software



La principal característica del movimiento DevOps es defender enérgicamente la automatización y la monitorización en todos los pasos de la construcción del software, desde la integración, las pruebas, la liberación hasta la implementación y la administración de la infraestructura

“you build it, you runs it”

Prácticas Devops

¿Cómo adquiere el equipo de desarrollo todo el conocimiento que aportan los silos de especialistas? Los pilares de la cultura devops son la automatización y las buenas prácticas. Estas son algunas prácticas que ayudan a adquirir una cultura devops:

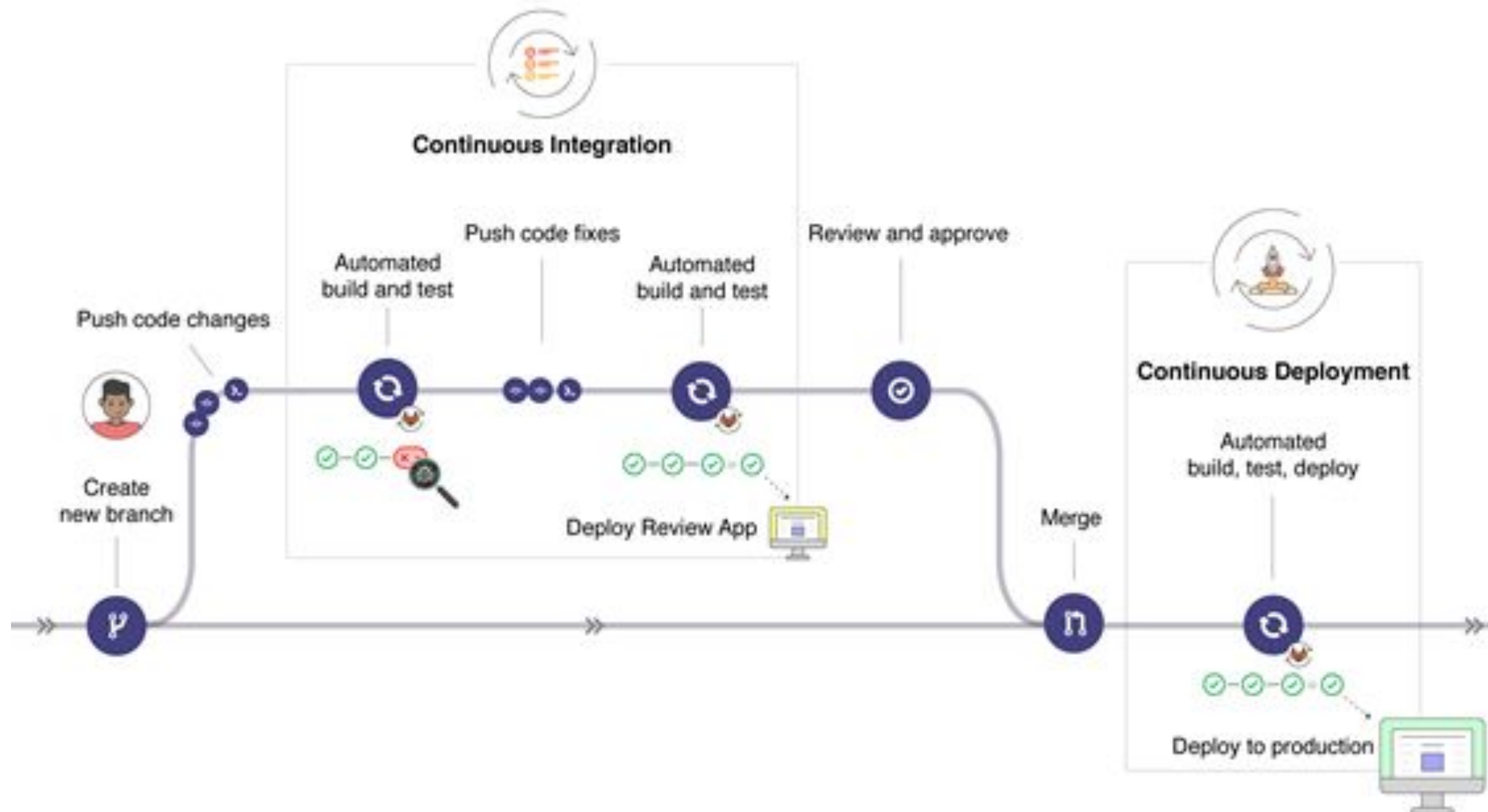
- ❑ **Gestión de la configuración.** Es necesario establecer una estrategia de ramas en el Sistema de Control de Versiones (SCV) que ayude a acortar los tiempos de despliegue reduciendo las mezclas de ramas.
- ❑ **Entrega continua.** El objetivo es hacer despliegues predecibles, automáticos y rutinarios en cualquier momento.
- ❑ **Infraestructura como código.** Una vez virtualizada la infraestructura es posible administrarla escribiendo código como si de un programa se tratase.
- ❑ **Contenedores.** La creación de contenedores es un enfoque de desarrollo de software en el que una aplicación o producto software, sus dependencias y su configuración se empaquetan en un archivo “contenedor”.
- ❑ **Microservicios.** El término “arquitectura de microservicios” ha surgido en los últimos años para describir una forma particular de diseñar aplicaciones de software como conjuntos de servicios funcionalmente autónomos y desplegados independientemente en contraposición con la “arquitectura monolítica”.
- ❑ **Monitorización.** Monitorizar el funcionamiento de una aplicación y registrar cómo el usuario se relaciona con dicha herramienta es crucial para entender cómo funciona, dónde falla y qué hay que mejorar. De esta forma, el equipo de desarrollo será mucho más eficiente a la hora de realizar cambios en el software o el sistema.
- ❑ **Comunicación y colaboración.** DevOps plantea un cambio cultural en las organizaciones que tiene como pilares la comunicación y cooperación entre Desarrollo y Operaciones.

Prácticas Devops

- » **Entrega Continua**
- » **Infraestructura como Código**
- » **Contenedores**
- » **Microservicios**

>> Entrega Continua

El **despliegue en producción** de los artefactos generados en el ciclo de integración continua se denomina “Continuous Delivery” (Entrega Continua).



Prácticas Devops

- » **Entrega Continua**
- » **Infraestructura como Código**
- » **Contenedores**
- » **Microservicios**

>> Infraestructura como Código

Infraestructura como código (Infraestructura as Code, IaC) o “**Infraestructura programable**” es la administración de la infraestructura (redes, máquinas virtuales, equilibradores de carga y topología de conexión) con un modelo descriptivo. Al igual que un mismo código fuente genera un mismo código binario, un archivo de definición de infraestructura generará la misma infraestructura cada vez que se ejecuta. IaC es una práctica clave de DevOps y se usa junto con la entrega continua.

```
resource "digitalocean_droplet" "web" {  
    image      = "ubuntu-18-04-x64"  
    name       = "domain.com"  
    region    = "nyc1"  
    size       = "s-1vcpu-1gb"  
}
```


Prácticas Devops

- » **Entrega Continua**
- » **Infraestructura como Código**
- » **Contenedores**
- » **Microservicios**

>> Contenedores

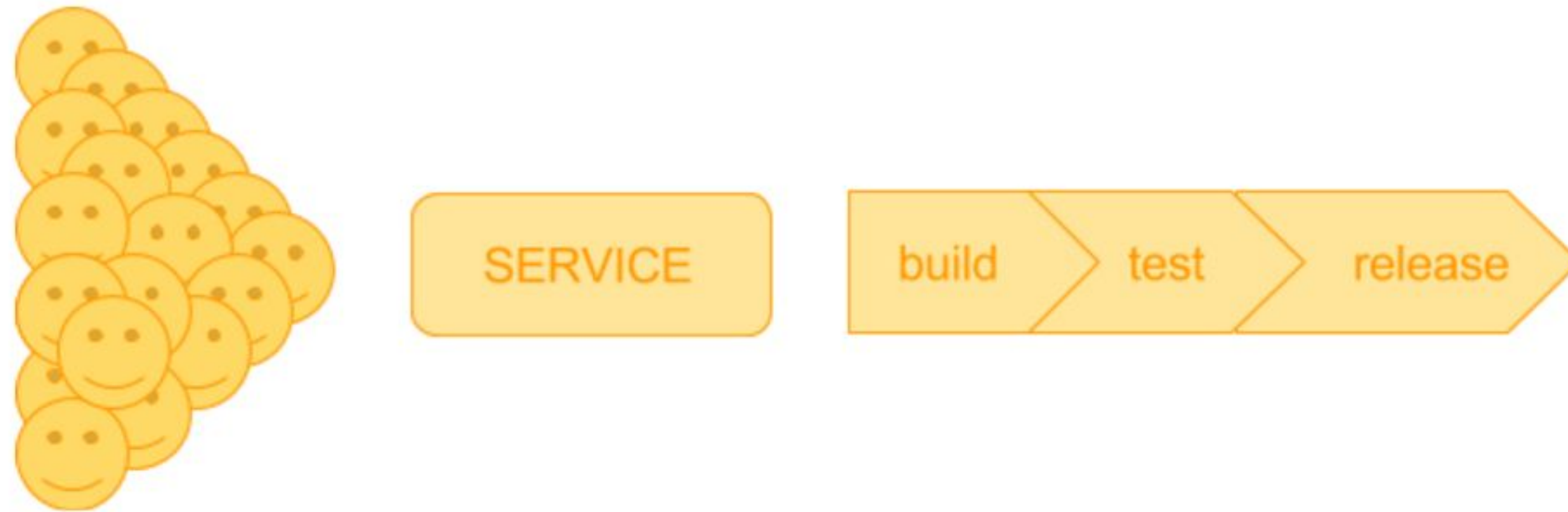
La creación de contenedores es un enfoque de desarrollo de software en el que una aplicación o producto software, sus dependencias y su configuración se empaquetan en un archivo “contenedor”. El archivo puede “*montarse*” en el sistema hospedador de contenedores, el cual se encargará de aportar todas las dependencias e infraestructura necesarias para el correcto funcionamiento de la aplicación.



Prácticas Devops

- » **Entrega Continua**
- » **Infraestructura como Código**
- » **Contenedores**
- » **Microservicios**

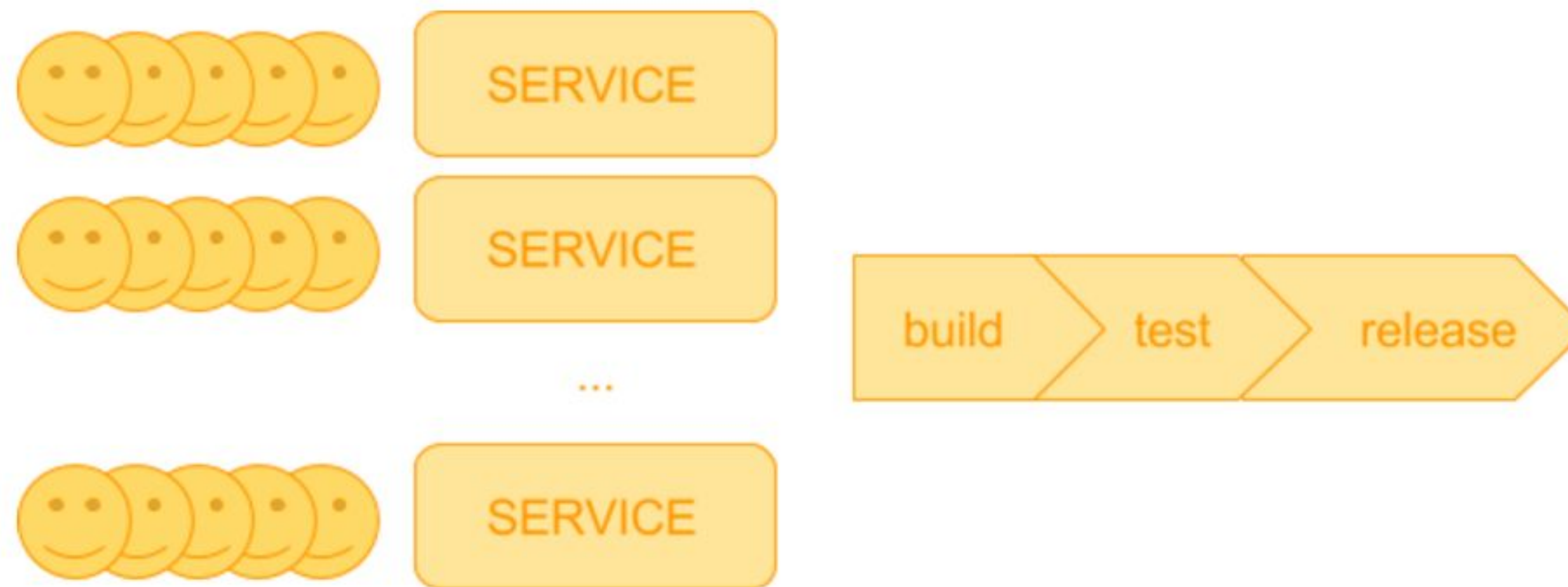
>> Microservicios



En 2001 el **Market de Amazon** se basaba en una arquitectura monolítica. A medida que la empresa creció, esta arquitectura sobre la que todos sus desarrolladores tenían que realizar modificaciones se volvió inmanejable. El equipo de trabajo rondaba los cien programadores y mezclar los *commits* (aportaciones de código) de todas ellas resultaba una tarea ardua y proclive a errores.

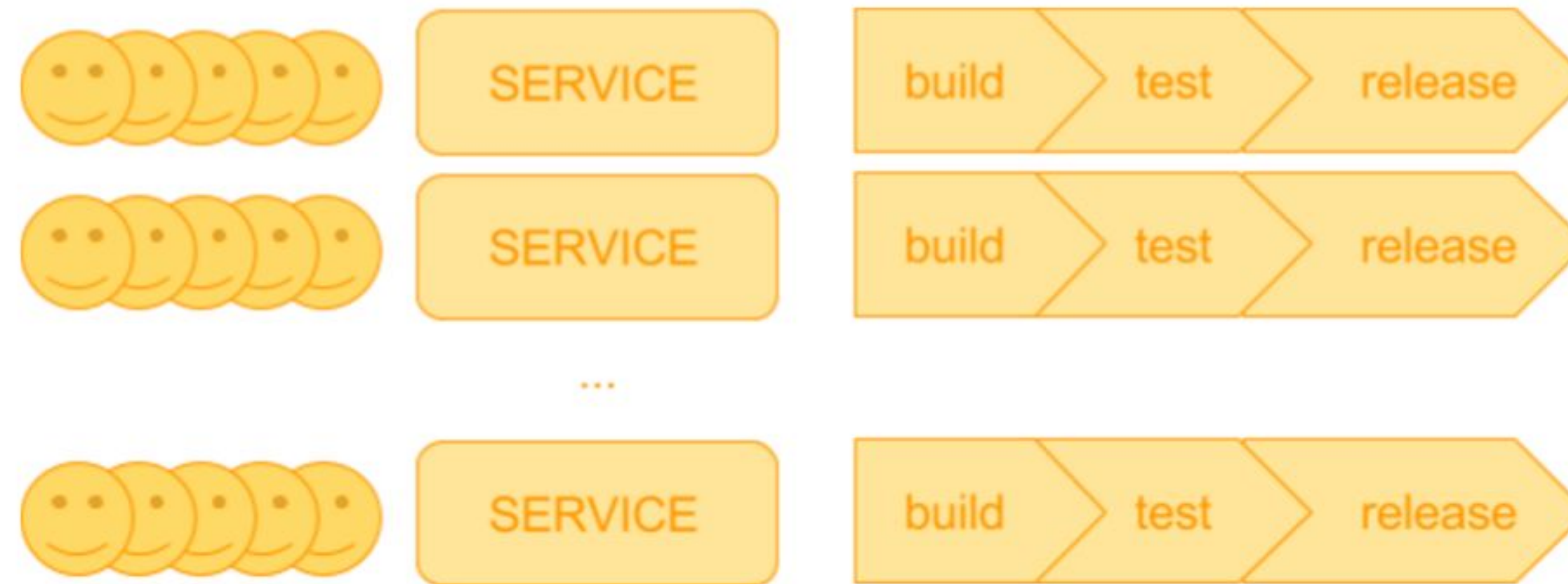
De esta forma, lo que en principio definieron como el *merge friday* pasó a convertirse en el *merge week* o en las *merge wars*. Esto dilató los intervalos temporales entre despliegues.

>> Microservicios



Amazon decidió dividir el código de la web en muchos pequeños servicios comunicados entre sí por sus APIs públicas. El equipo de cien personas se dividió en muchos equipos de aproximadamente ocho personas, cada uno de ellos responsable de un microservicio. Esta estrategia ayudó a desacoplar el código reduciendo los conflictos al mezclarlo.

>> Microservicios



Uno de los problemas que tenía Amazon es que había gran cantidad de equipos de desarrollo, pero sólo un equipo de despliegue. Aquí es donde entra en acción la cultura **Devops**. Para poder gestionar tal cantidad de microservicios era necesaria una herramienta “self service” para automatizar el proceso de despliegue cada vez que uno de los equipos incluyera cambios. Tenía que ser agnóstica de plataforma, con gestión de versiones, que incluyera un testeo previo (tests unitarios) y posterior (tests funcionales) al despliegue y, a ser posible, que pudiera gestionar automáticamente el rollback de la aplicación en caso de error. Tras automatizar el proceso de despliegue, cada uno de los microservicios dispuso de su propio pipeline de despliegue.

